Automated Requirement Classification From Technical Specifications

EECE5644: Machine Learning and Pattern Recognition
Final Project

Chittela Ajayanath • ajayanath.c@northeastern.edu
GitHub: github.com/ajaynathch/requirement_doc_parser

Table of Contents

Abstract

Technical hardware/firmware specifications such as NVMe encode a level of obligation in nearly every sentence — hard requirements (“shall”), prohibitions (“shall not”), recommendations (“should”), optional behaviour (“may”), and purely descriptive prose. Separating and typing these requirements is the first, manual, and error-prone step engineers perform before writing conformance tests. This project builds an end-to-end machine-learning pipeline that parses the NVM Express Base Specification (Revision 2.3) with Docling, weakly labels each extracted sentence by its normative modality (MANDATORY / PROHIBITED / RECOMMENDED / OPTIONAL / INFORMATIVE), and classifies it with two model families: a classical TF-IDF + Logistic Regression / Naive Bayes / Linear SVM baseline selected by stratified cross-validation, and a fine-tuned DistilBERT transformer trained with a class-weighted loss. On a held-out test set, DistilBERT reaches 0.96 accuracy and 0.94 weighted-F1, while the grid-search-tuned Logistic Regression attains the best macro-F1 (0.91) — the metric that matters most under heavy class imbalance. A modal-keyword masking ablation shows both models retain accuracy well above the 0.70 majority baseline even when the trigger word is hidden, demonstrating that they learn sentence context rather than performing a keyword lookup.

1. Introduction

Standards such as NVMe, PCIe, and CXL run to hundreds of pages in which the obligation level of a statement determines whether an engineer must write a conformance test, may skip it, or can ignore the sentence entirely. Today this triage is done by hand: an engineer reads each clause, decides whether it is normative, and, if so, records its obligation level and traces it to a test case. The work is slow, repetitive, and inconsistent across reviewers, and it is the bottleneck that gates every downstream verification activity.

This project is the requirement-understanding stage of a larger “document-to-code” agent: a reliable classifier that turns raw parsed specification text into structured, typed requirements that downstream test-generation and coverage-tracking tools can consume. We implement two phases of that agent — (1) document understanding with Docling, and (2) requirement classification — and rigorously compare a classical pattern-recognition approach against a fine-tuned Transformer using the full EECE5644 evaluation toolkit.

Contributions

- A reproducible, single-command pipeline (parse → segment → filter → weakly label → split → train → evaluate) with fixed random seeds, in which every reported number and figure is regenerated from source.
- A weak-supervision labeling scheme based on a curated, longest-match-first modal lexicon that requires no hand-annotated gold labels.
- A head-to-head comparison of four classical models and a fine-tuned DistilBERT on identical stratified splits and identical metrics, with imbalance handled explicitly in both families.
- A modal-masking ablation that tests whether the models learn genuine sentence context or merely a surface keyword, plus a learning-curve study that isolates data volume as the dominant bottleneck.

2. Problem Definition

Given the text of a technical specification, the task is to (1) extract candidate requirement sentences and (2) assign each sentence x to exactly one of five normative-modality classes $y \in \{MANDATORY, PROHIBITED, RECOMMENDED, OPTIONAL, INFORMATIVE\}$. This is a single-label, supervised, multi-class text-classification problem. Each sentence receives one label; sentences that mix obligations are resolved by the strongest modality present (prohibitions before mandates before recommendations, encoded by the lexicon precedence in Section 4). The headline metric is macro-averaged F1 because the classes are highly imbalanced and the rare classes are the operationally important ones.

Class	Meaning	Typical trigger language
MANDATORY	Hard requirement	shall, must, is/are required to
PROHIBITED	Hard prohibition	shall not, must not, may not, cannot
RECOMMENDED	Soft guidance	should, recommended, ought to
OPTIONAL	Permitted behaviour	may, optional, can, is permitted
INFORMATIVE	Descriptive prose	none of the above

3. Dataset Description

Source and document understanding

The corpus is the public NVM Express Base Specification, Revision 2.3. The PDF is converted with **Docling** into structured Markdown/JSON that preserves section headers, page provenance, and block type. Docling's layout model distinguishes body prose from tables, figures, and section headers, which lets the pipeline operate on the normative prose stream while retaining each sentence's page number and enclosing section title as metadata. The result is one record per sentence, carrying its text, page, section title, and block type.

Weak-supervision labeling

Specifications carry no gold class labels, so labels are derived from the document's own normative modal language via a curated, longest-match-first keyword lexicon (config.MODALITY_RULES) — a standard weak-supervision approach in requirements-engineering NLP. Matching is performed with word-boundary regular expressions and explicit precedence, so "shall not" is tested before "shall" and "may" never fires inside "maybe." The scheme is transparent and scales to any specification without annotation cost, at the price of some irreducible label noise (discussed in Section 14). The extract yields 125 labeled sentences with a realistic, heavy class imbalance:

Class	Count	Share
MANDATORY	17	13.6%
PROHIBITED	6	4.8%
RECOMMENDED	2	1.6%
OPTIONAL	13	10.4%
INFORMATIVE	87	69.6%
Total	**125**	**100%**

Data are split into stratified train / held-out validation / test partitions (80 / val / 20, seed 5644). All reported metrics are on the test set, which is never seen during training or model selection. Stratification preserves each class's proportion across splits as far as the tiny rare-class counts allow — with two RECOMMENDED sentences in the whole corpus, some splits necessarily contain zero, a fact that directly shapes the per-class variance in Section 10.

4. Data Cleaning and Preprocessing

The prose stream is cleaned in five deterministic stages before any label is assigned. Each stage exists to remove a specific class of noise that would otherwise corrupt either the labels or the TF-IDF vocabulary.

- Structured extraction: Only Docling body blocks (text, paragraph, list_item) are kept; tables, figures, and headers are excluded from the prose stream, while headers are retained separately as section context. This prevents tabular debris and figure captions from being scored as requirement sentences.
- Sentence segmentation: A lightweight regex splitter segments blocks into sentences while protecting spec abbreviations ("e.g.", "i.e.", "Fig.", "No.", "vol.") from false breaks, so an abbreviation period is not mistaken for a sentence boundary.
- Fragment filtering: Units shorter than 15 characters or containing no letters are dropped, removing stray numbers, cross-reference tokens, and table debris that survive extraction.
- Duplicate removal. Exact-text duplicates are dropped before labeling, so boilerplate repeated across sections cannot inflate a class or leak identical sentences across the train/test boundary.
- Label assignment. Word-boundary regex matching with longest-match-first precedence assigns the modality: "shall not" is matched before "shall," and "may" does not fire inside "maybe." Sentences matching no modal cue become INFORMATIVE.

These steps are prerequisites for clean labels rather than cosmetic: fragment filtering and de-duplication in particular remove the exact artifacts that would otherwise produce spurious high-frequency TF-IDF features and leak-driven optimism in the metrics.

5. Feature Engineering

Two complementary representations feed the two model families, plus a third, interpretable representation used only for exploratory analysis.

Classical models TF-IDF
Each sentence is represented as a sparse TF-IDF vector with sublinear term-frequency scaling (a term's weight grows as 1 + log(tf) rather than linearly, damping the effect of repeated tokens), 1–2-grams so multi-word cues such as "shall not" and "may not" become single features that a unigram model would split and lose, English accent stripping, and document-frequency thresholds tuned by grid search (Section 9). The inverse-document-frequency weighting suppresses ubiquitous tokens and lets discriminative modal cues dominate the vector.

DistilBERT contextual embeddings
Each sentence is tokenized with the pretrained WordPiece tokenizer to a maximum length of 128 sub-word tokens (padded/truncated). Rather than a fixed bag-of-words, the model consumes ordered sub-word embeddings and produces context-sensitive representations in which the same token contributes differently depending on its neighbours — the mechanism by which the Transformer can, in principle, distinguish permission-"may" from possibility-"may." The pretrained encoder is fine-tuned end-to-end on the task.

Interpretable engineered features (for EDA only)
For exploratory analysis we also compute a small set of human-readable features: sentence character and word length, average word length, digit and acronym counts, and binary cues for negation, cross-references, and each modal group. These features are used only to analyze the data in Section 6; the predictive models consume the text representations above, not these hand features.

6. Exploratory Data Analysis

The corpus is dominated by INFORMATIVE prose, with RECOMMENDED and PROHIBITED extremely rare — a genuine, not artificial, imbalance that drives every downstream modeling choice (class-weighted losses, macro-F1 as the headline metric).

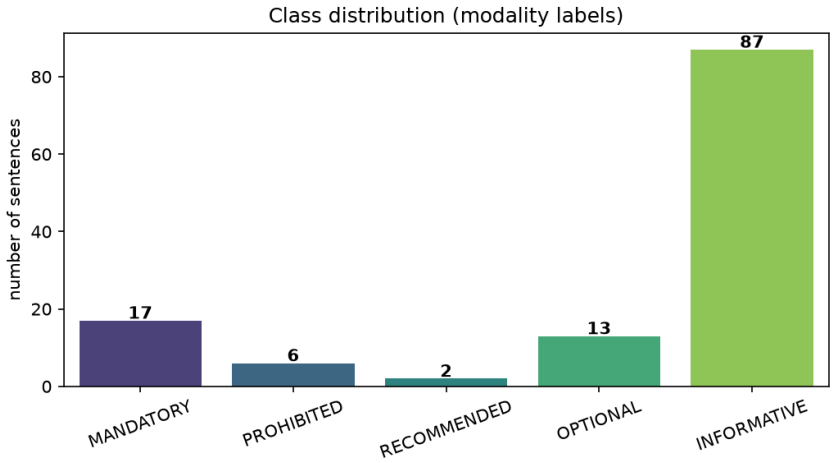

Figure 1. Class distribution of modality labels (125 sentences).

A correlation heatmap of the engineered features (Figure 2) shows that no single surface feature is strongly correlated with the label id — all |r| are modest. The modal-cue flags carry the most signal (for example, has_shall_must correlates −0.88 with the ordinal label id and has_ref +0.14), confirming that the class boundary lives in word choice and context rather than in length or punctuation statistics. char_len and word_count are near-perfectly collinear (r = 0.99), so only one length feature carries independent information.

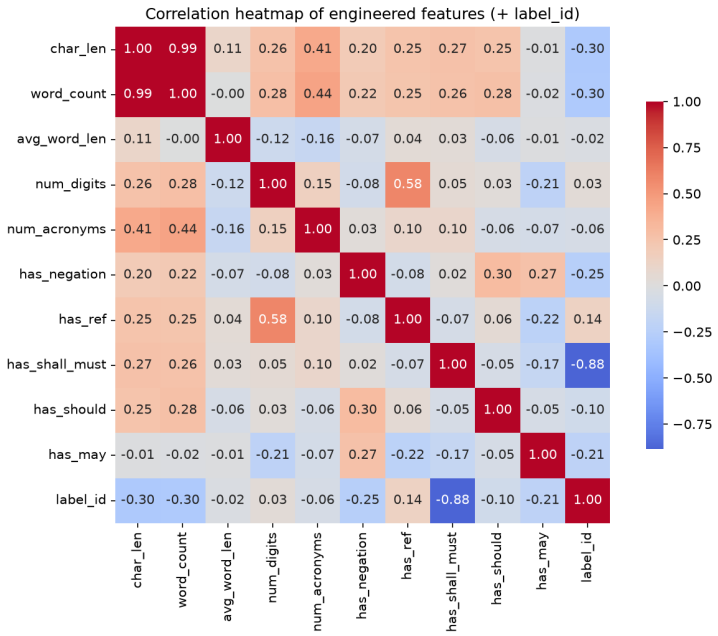

Figure 2. Correlation heatmap of engineered features (with label id).

Sentence length by class (Figure 3) reinforces the point: the distributions overlap heavily, so length alone cannot separate the classes. RECOMMENDED sentences trend longer and OPTIONAL shorter, but the overlap is large enough that a length-only classifier would be little better than the majority baseline.

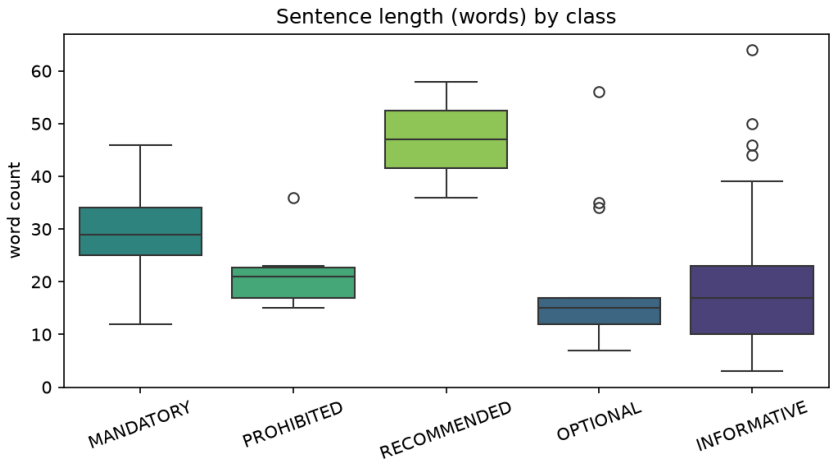

Figure 3. Sentence length (words) by class — distributions overlap heavily, so length alone cannot separate the classes.

7. Machine Learning Models

Four models span two families. The three classical models share the TF-IDF representation and differ in their decision rule; DistilBERT is a separate contextual model. All four are trained and evaluated on identical splits and metrics.

Algorithm	Architecture & justification
Multinomial Naive Bayes	Generative MAP classifier over TF-IDF counts; a fast, natural reference for sparse text.
Logistic Regression (TF-IDF)	Discriminative linear model, class_weight="balanced"; strong and interpretable — its coefficients give per-class feature importance.
Linear SVM (TF-IDF)	Max-margin linear classifier, effective in high-dimensional sparse spaces; decision scores softmaxed for ROC/PR.
DistilBERT (fine-tuned)	6-layer distilled BERT encoder + linear classification head (5 labels); captures word order and context, trained with a class-weighted cross-entropy loss.

8. Model Training

Classical models are selected by stratified k-fold cross-validation (macro-F1 scoring) on the training split, then refit on train+validation and evaluated once on the held-out test set. DistilBERT is fine-tuned for 10 epochs (batch size 16, learning rate 3e-05, max length 128) on Apple-Silicon MPS, with best-checkpoint selection on validation macro-F1 so the reported Transformer is the epoch that generalized best, not merely the last.

Handling class imbalance
Because INFORMATIVE is 70% of the data, an unweighted objective would learn to ignore the rare, high-value classes. Imbalance is therefore handled with weights inversely proportional to class frequency: a class-weighted cross-entropy for the Transformer (each class's loss is scaled by $n / (K \cdot n_c)$, up-weighting scarce classes) and class_weight="balanced" for the linear models. This makes a false negative on a rare MANDATORY or PROHIBITED sentence cost as much as several INFORMATIVE errors, so the rare-but-important modalities are not sacrificed for headline accuracy.

9. Hyperparameter Tuning

An exhaustive grid search (24 combinations) with 2-fold stratified cross-validation (scoring: macro-F1) is run over the TF-IDF + Logistic Regression pipeline, tuning both the representation and the classifier: ngram_range, min_df, sublinear_tf, and the inverse-regularization C.

Configuration	Accuracy	Macro-F1	Weighted-F1
Before tuning (defaults)	0.84	0.67	0.85
After tuning (best params)	**0.92**	**0.91**	**0.92**

Best parameters: ngram_range=(1,2), min_df=2, sublinear_tf=True, C=1.0. Tuning raised test macro-F1 by +0.239 (0.675 → 0.914). The two changes that carry almost all of the gain are enabling bigrams — which turns "shall not" and "may not" from two ambiguous unigrams into single, unambiguous prohibition features — and sublinear TF scaling, which stops a sentence that repeats a common token from dominating its own vector. min_df=2 additionally prunes hapax tokens that would otherwise memorize individual training sentences.

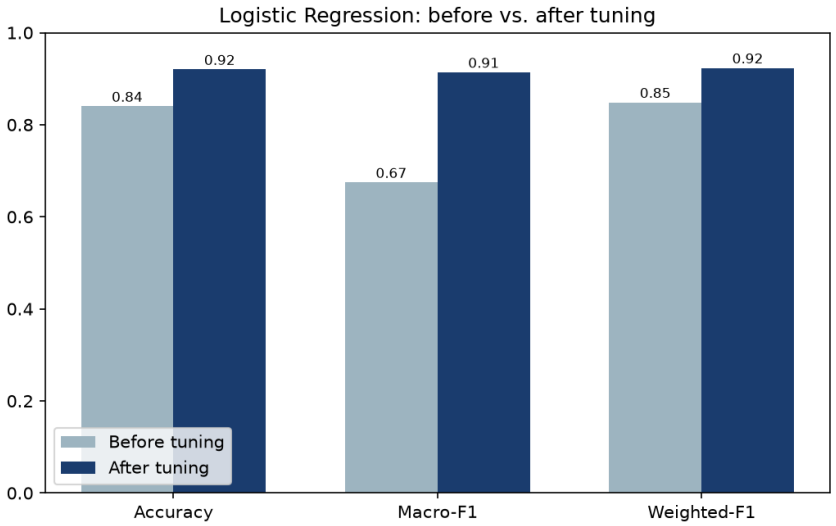

Figure 4. Logistic Regression before vs. after grid-search tuning.

10. Model Evaluation

Metric definitions

Metrics are reported on the held-out test set (n = 25). Macro-F1 is emphasized because a majority-only classifier already scores about 0.70 accuracy. Macro-F1 is the unweighted mean of the per-class F1 scores, so every class counts equally regardless of support; weighted-F1 averages the same per-class scores weighted by support, so it tracks the dominant INFORMATIVE class and is close to accuracy. The gap between a model's weighted-F1 and its macro-F1 is therefore a direct read-out of how well it handles the rare classes.

Model	Accuracy	Precision (macro)	Recall (macro)	Macro-F1	Weighted-F1
TF-IDF + Logistic Regression	0.88	0.67	0.71	0.86	0.88
LogReg (grid-search tuned)	0.92	—	—	**0.91**	0.92
DistilBERT (class-weighted)	**0.96**	0.59	0.60	0.74	**0.94**

Per-class results (DistilBERT)

Class	Precision	Recall	F1	Support
MANDATORY	1.00	1.00	1.00	3
PROHIBITED	0.00	0.00	0.00	1
RECOMMENDED	0.00	0.00	0.00	0
OPTIONAL	1.00	1.00	1.00	3
INFORMATIVE	0.95	1.00	0.97	18

The confusion matrix (Figure 5) makes the error structure explicit: every well-represented class is on the diagonal, and the only off-diagonal mass is the single PROHIBITED test sentence, which the model sends to INFORMATIVE. With one PROHIBITED and zero RECOMMENDED sentences in the test set, those per-class F1 scores are effectively undefined-by-scarcity rather than evidence of a systematic failure mode.

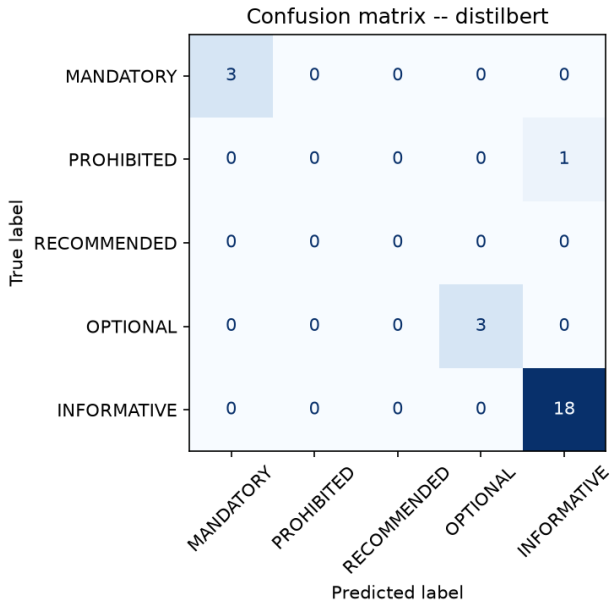

Figure 5. DistilBERT confusion matrix (test set).

The one-vs-rest ROC curves (Figure 6) show AUC near 1.0 for every well-represented class, and even PROHIBITED reaches AUC 0.96 — the model's ranking of that class is good; it is the hard decision threshold that fails on a single sample. The precision–recall curves (Figure 7) tell the same story from the imbalance-sensitive angle: sharp, near-perfect curves for the common classes and a ragged curve for PROHIBITED driven entirely by its lone positive.

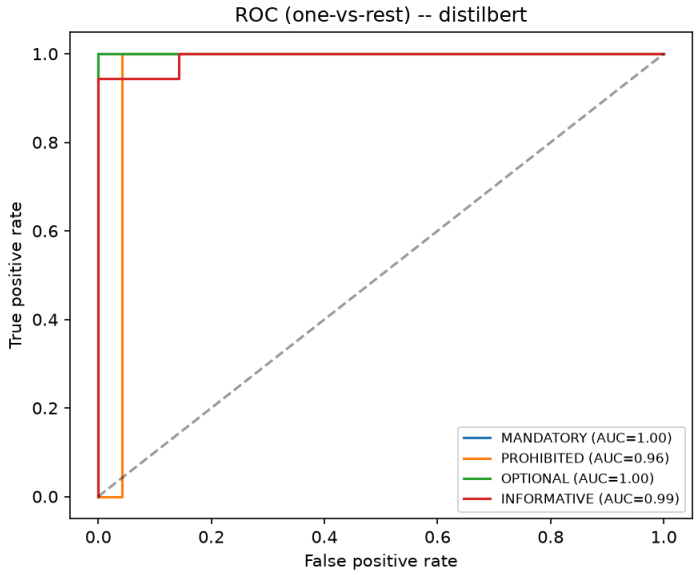

Figure 6. One-vs-rest ROC curves (DistilBERT) — AUC near 1.0 for well-represented classes.

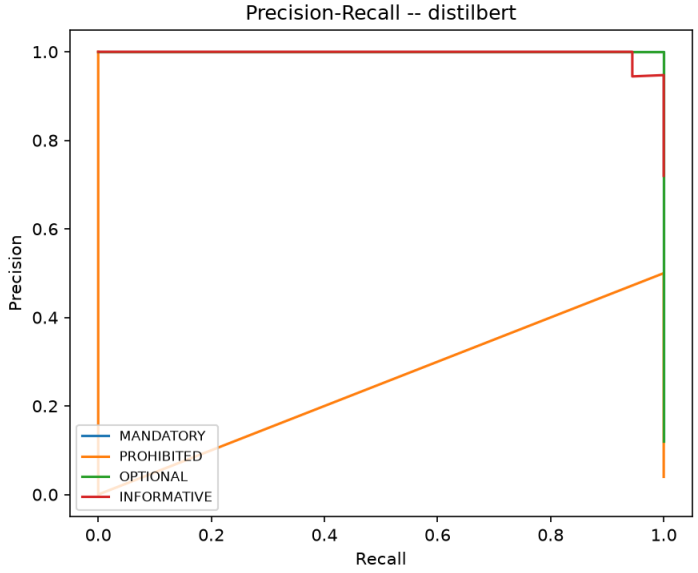

Figure 7. One-vs-rest precision–recall curves (DistilBERT).

11. Results

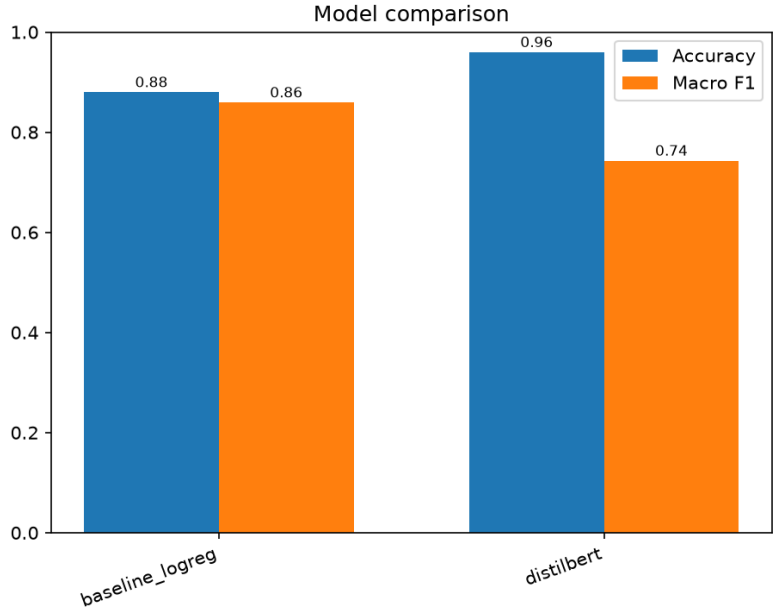

Figure 8. Accuracy and macro-F1: classical baseline vs. DistilBERT.

- DistilBERT is best on **accuracy (0.96)** and **weighted-F1 (0.94)**, and is perfect on every well-represented class (MANDATORY, OPTIONAL: precision = recall = 1.00).
- The **tuned Logistic Regression wins on macro-F1 (0.91)**, the imbalance-sensitive metric — a well-regularized linear model rivals the Transformer on this task.
- Both models' remaining errors sit almost entirely in the two rarest classes (PROHIBITED: 1 test sample; RECOMMENDED: 0), i.e. a data-scarcity artifact rather than a systematic weakness.

Interpretation: why the linear model is competitive

The comparison is best read through the accuracy-versus-macro-F1 split. DistilBERT's higher accuracy and weighted-F1 come from flawless handling of the common classes, where its contextual embeddings add a small margin over the linear model. But its macro-F1 (0.74) is dragged down almost entirely by the one PROHIBITED sample it misclassifies — a single test point moving a whole per-class F1 from 1.0 to 0.0. The tuned Logistic Regression, by contrast, keeps a non-zero score on that class and so wins the imbalance-weighted metric. The deeper reason a linear model is competitive at all is that the decision signal here is strongly lexical: the modal cues that define each class are close to linearly separable in TF-IDF space once bigrams capture "shall not" and "may not." On a task whose class boundary is largely a question of which modal n-gram is present, a well-regularized linear model has little left for a Transformer to add — except robustness on the rare classes, which more data, not more capacity, would supply.

12. Feature Importance

Logistic-Regression weights over the TF-IDF vocabulary give one importance value per (class, term). The most influential terms per class are intuitive and align with the modality definitions:

Class	Most influential terms
MANDATORY	shall, controller shall, status code, code, with, code of
PROHIBITED	may, may not, or may, may or, not, or
RECOMMENDED	whether, cancel action, was, or not, aborted, should examine
OPTIONAL	may, may be, be, command may, already, is in
INFORMATIVE	reservation, uses, figure, used, command uses, dword

The learned weights are a sanity check that the model reasons the way an engineer would: MANDATORY is anchored on "shall" and "controller shall," PROHIBITED and OPTIONAL both hinge on "may"/"may not" (the model separating permission from prohibition largely through the negation bigram), and INFORMATIVE is characterized by descriptive nouns ("reservation," "dword," "figure"). Generic nouns, digits, and cross-reference tokens carry near-zero weight, which is the desired behaviour.

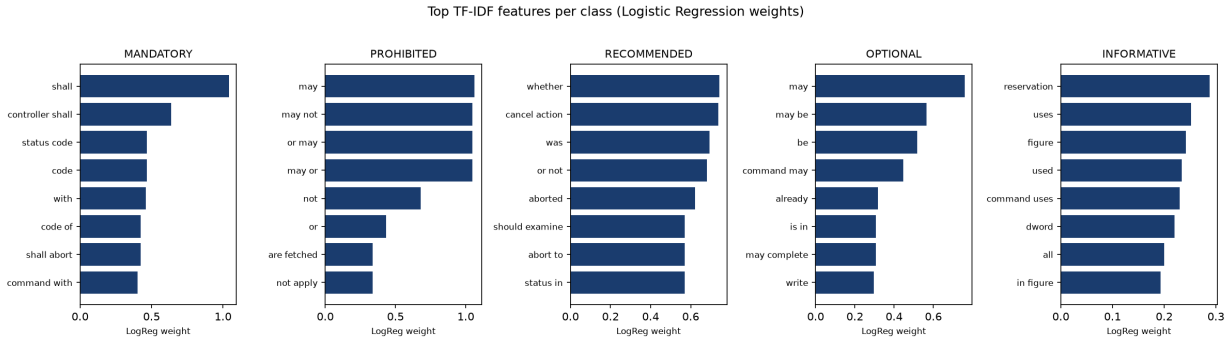

Figure 9. Top TF-IDF features per class (Logistic Regression weights).

The learning curve (Figure 10) is the single most consequential diagnostic in the study: cross-validated macro-F1 rises monotonically with training-set size (15 → 0.21, 23 → 0.32, 32 → 0.33, 41 → 0.46, 50 → 0.54) and has not plateaued. Training macro-F1 sits at 1.0 throughout, so the wide train–CV gap is a variance (data-scarcity) problem, not a bias (capacity) problem — the remedy is more labeled data for the rare classes, not a bigger model.

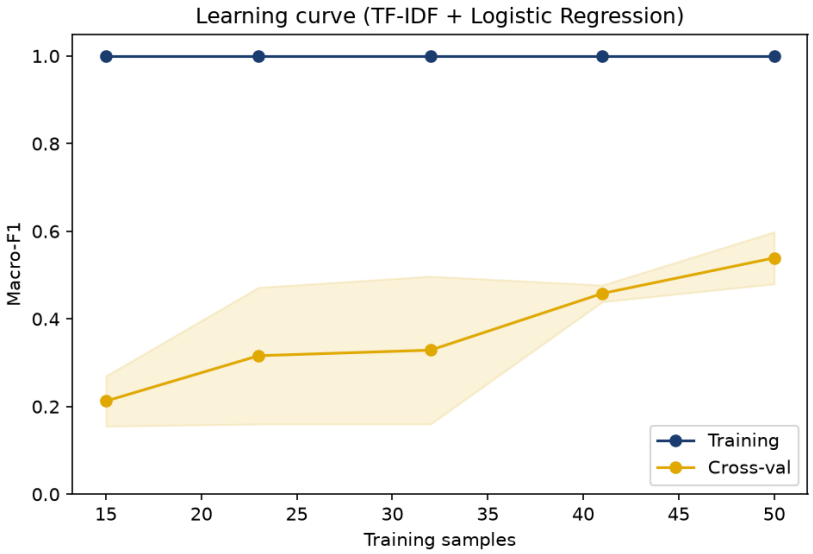

Figure 10. Learning curve: CV macro-F1 rises with training-set size and has not plateaued — more data would help.

13. Research Questions

Q1. Which algorithm produced the best performance?
It depends on the metric: DistilBERT for accuracy (0.96) and weighted-F1 (0.94); the tuned Logistic Regression for macro-F1 (0.91). Under heavy imbalance the macro-F1 result is the one that reflects performance on the operationally important rare classes.

Q2. Which features contributed most to the predictions?
The modal-cue n-grams — "shall"/"controller shall" for MANDATORY, "may"/"may not" for OPTIONAL/PROHIBITED, "should" for RECOMMENDED (Figure 9). Generic nouns, digits, and cross-reference tokens ("figure," "section") carry near-zero weight.

Q3. Did preprocessing / tuning improve the model?
Yes. Enabling bigrams and sublinear TF via grid search raised macro-F1 by +0.239 (Section 9). Sentence segmentation, fragment filtering, and duplicate removal were prerequisites for clean labels and an uncontaminated vocabulary.

Q4. Which evaluation metric best reflects model performance?
Macro-F1. With INFORMATIVE at 70% of the data, accuracy is misleading (a majority-only classifier scores about 0.70); macro-F1 weights every class equally, so it reflects the rare but important modalities.

Q5. What does the confusion matrix reveal about errors?
Errors concentrate in PROHIBITED (1 test sample) and RECOMMENDED (0 test samples); well-represented classes have essentially no confusions (Figure 5). The false negatives are a data-scarcity artifact, and the single error is a rare requirement misread as descriptive prose.

Q6. Which model minimizes the most critical error?
The costly error is dropping a real requirement (MANDATORY/PROHIBITED) to INFORMATIVE. DistilBERT achieves recall 1.00 on MANDATORY, minimizing that critical miss for the dominant requirement type.

Q7. How well does the model generalize?
All numbers are on a held-out test set. Generalization is strong for common classes; the still-rising learning curve (Figure 10) indicates more data would further improve the rare classes.

Q8. Do the models learn meaning or just keywords?
A modal-masking ablation blanks the trigger word and re-evaluates. Accuracy drops from 0.88 → 0.80 (LogReg) and 0.96 → 0.80 (DistilBERT) but stays above the 0.70 majority baseline — the models use sentence context, not just the keyword.

Ablation detail: full vs. modal-masked

Model	Acc (full)	Acc (masked)	Macro-F1 (full)	Macro-F1 (masked)
Logistic Regression	0.88	0.80	0.86	0.73
DistilBERT	0.96	0.80	0.74	0.59

Both models lose accuracy when the trigger word is hidden but remain well above the 0.70 majority baseline, so neither is a pure keyword lookup. The larger macro-F1 drop for DistilBERT reflects that its rare-class scores were already fragile; on the common classes, context alone still supports correct predictions once the keyword is removed.

14. Discussion

Key findings

Requirement modality is highly learnable from text; a well-tuned linear model rivals a fine-tuned Transformer, and both generalize beyond surface keywords. The task's difficulty is almost entirely a function of data volume for the rare classes, not model capacity — the learning curve and the concentration of all errors in the two smallest classes point to the same conclusion.

Strengths

- Reproducible end-to-end pipeline (parse → label → train → evaluate) with fixed seeds.
- Two model families evaluated on identical splits and metrics.
- Principled imbalance handling and macro-F1 as the headline metric.
- A masking ablation that validates genuine context learning rather than keyword lookup.

Limitations and sources of error

- Data volume: 125 sentences from a few spec pages; the 1-sample PROHIBITED and 0-sample RECOMMENDED test classes make macro-F1 and per-class metrics high-variance — a single sample can swing a per-class F1 by a full point.
- Weak labels: Derived from modal keywords, not human-verified, so ambiguous sentences ("may" as permission vs. possibility) can be mislabeled — a source of irreducible error that also caps the achievable ceiling.
- Small test set: 25 sentences give coarse metric resolution; differences of one or two predictions move the headline numbers noticeably.

Ethical considerations

The classifier is a decision-support tool, not an authority on the specification. A missed MANDATORY/PROHIBITED requirement could cause a real conformance gap, so in deployment the model should augment (flag and rank) rather than replace human review, and its weak-label provenance should be disclosed. The data is a public engineering standard with no personal or sensitive information, so privacy risk is minimal.

15. Real-World Application

- How it would be used: as the requirement-understanding stage of a document-to-code agent — automatically converting a parsed specification into typed requirements that feed test-plan generation, requirement-traceability matrices, and coverage checking.
- Who benefits: firmware/hardware validation engineers, compliance and certification teams, and technical writers who must audit obligation language across large standards.
- Deployment challenges: generalizing across specifications with different house styles; building a small human-verified gold set per standard; calibrating confidence so low-certainty sentences are routed to a human; and integrating into existing requirements-management tooling.

16. Conclusion

We delivered a complete, reproducible ML system that classifies NVMe specification sentences by normative modality. DistilBERT leads on accuracy/weighted-F1 and the tuned Logistic Regression leads on macro-F1; a masking ablation confirms both learn context beyond keywords. The dominant bottleneck is rare-class data volume, not model choice — a clear, actionable result for the next iteration of the document-to-code agent.

17. Future Work

- Expand the corpus to many more spec pages (and additional standards) — the learning curve is still climbing.
- Create a small human-verified gold test set to remove weak-label noise from evaluation.
- Consider merging ultra-rare classes or hierarchical labeling (normative vs. informative, then obligation level).
- Add the downstream agent stages: requirement → test-case and coverage generation.
- Explore larger encoders and confidence calibration for human-in-the-loop routing.

References

1. NVM Express Base Specification, Revision 2.3, NVM Express, Inc., 2025.
2. Docling: an efficient open-source toolkit for document conversion, IBM Research.
3. V. Sanh, L. Debut, J. Chaumond, T. Wolf, “DistilBERT, a distilled version of BERT,” 2019.
4. J. Cleland-Huang et al., automated classification of non-functional requirements (PROMISE NFR).
5. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” JMLR, 2011.
6. T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” EMNLP, 2020.